

Acceso a Datos Sesión 24-09-2025

Enfoque de Acceso a Datos en Angular

1. Diferencia clave respecto a Java/PHP/Python

- En **Java/PHP/Python**, el acceso a datos se hace directamente contra ficheros o BBDD.
- En **Angular**, el cliente **NO accede directamente a la base de datos** → lo hace a través de un **backend (API REST o GraphQL)**.
- Angular trabaja con:
 - **Servicios HTTP (HttpClient)** para consumir APIs.
 - **Observables (RxJS)** para manejar asincronía.
 - **Modelos/Interfaces** para tipar los datos (TypeScript).

2. Contenidos a trasladar

Contenido Acceso a Datos (DAM)	Equivalente en Angular
Ficheros (lectura/escritura)	Uso de <code>FileReader</code> , <code>Blob</code> , subida/descarga de ficheros vía HTTP.
Bases de datos relacionales / NoSQL	Se consumen mediante API REST/GraphQL que el backend expone. Angular trabaja con servicios y modelos de datos .
ORM (Hibernate, JPA)	Angular no tiene ORM, pero puede usar librerías de tipado/serialización (<code>class-transformer</code> , <code>zod</code>) para convertir JSON a objetos.
Transacciones, concurrencia	Se gestionan en backend, pero Angular puede implementar manejo de errores y retry con RxJS .
Persistencia local	Angular ofrece LocalStorage , SessionStorage o IndexedDB para datos temporales.
Seguridad	Manejo de tokens JWT , cabeceras HTTP, interceptores para autenticar cada petición.

3. Módulos

Módulo 1 – Acceso a datos estáticos (JSON local)

- Cargar un fichero `data.json` con `HttpClient`.
- Mostrarlo en una tabla con Angular Material.
- Explicar **interfaces TypeScript** como modelo de datos.

```
export interface Ciudad {  
  id: number;  
  nombre: string;  
}
```

Servicio:

```
@Injectable({ providedIn: 'root' })  
export class CiudadService {  
  constructor(private http: HttpClient) {}  
  
  getCiudades(): Observable<Ciudad[]> {  
    return this.http.get<Ciudad[]>('assets/ciudades.json');  
  }  
}
```

Módulo 2 – Acceso a datos vía API REST

- Conectar Angular a un backend (ej. API en Node, Django o Spring Boot).
- CRUD completo (GET, POST, PUT, DELETE).
- Uso de **RxJS** (map, catchError) para procesar respuestas.

Ejemplo de alta de ciudad:

```
addCiudad(ciudad: Ciudad): Observable<Ciudad> {  
  return this.http.post<Ciudad>('http://localhost:3000/ciudades', ciudad);  
}
```

Módulo 3 – Persistencia local

- Guardar datos temporalmente en **LocalStorage**.
- Explicar que **no es seguro ni permanente** como una BBDD real.
- Ejemplo: guardar el último filtro de búsqueda.

```
localStorage.setItem('filtro', 'Madrid');  
const filtro = localStorage.getItem('filtro');
```

Módulo 4 – Seguridad en acceso a datos

- Introducir **JWT**: el backend devuelve un token al hacer login.
- Angular almacena el token en **LocalStorage/SessionStorage**.
- Cada petición HTTP incluye el token (con **interceptor**).

Módulo 5 – Ejercicios prácticos

- Subida y descarga de ficheros (`multipart/form-data`).
- Paginación y filtrado desde la API.
- Optimización con **cache en memoria** usando `BehaviorSubject`.
- Uso de **GraphQL** en vez de REST (si el centro lo permite).

4. Proyecto

- Aplicación Angular que gestione **ciudades**:
 - Alta, baja, modificación y listado.
 - Conexión a una **API REST** (Node/Express o Spring Boot).
 - Persistencia temporal en **LocalStorage** cuando no hay conexión.
 - Manejo de autenticación con **JWT**.

El objetivo es crear una pequeña app Angular que consuma un *backend falso* con **json-server**, permita añadir ciudades (sin duplicados, **ignorando mayúsculas/minúsculas**), listarlas, borrarlas y **exportarlas a CSV** desde el cliente. Incluiré todos los comandos, los ficheros clave y el código (servicio, componentes, modelo), y notas pedagógicas para que lo uses en clase.

Resumen de la arquitectura:

Cliente Angular (HTTP via `HttpClient`) $\longleftrightarrow$ json-server (fichero `db.json` que actúa como “base de datos” persistente en JSON).

Requisitos (preparación del entorno)

1. **Node.js (LTS)** y **npm** instalados (Angular CLI usa Node/npm).

Instalar Angular CLI (una vez):

```
npm install -g @angular/cli
```

2. (o usar `npmx` para ejecutar `ng` sin instalar globalmente).
3. Usaremos `json-server` como backend de pruebas (no hace falta programar un servidor real): se puede ejecutar con `npmx json-server --watch db.json --port 3000`.

Paso 0 — Crear el proyecto Angular

En una carpeta de trabajo:

```
# crear proyecto Angular (puedes cambiar el nombre 'ciudades-app')
ng new ciudades-app --routing=false --style=css
cd ciudades-app
```

(El `ng new` crea el workspace + app; puedes contestar las preguntas por defecto).

Paso 1 — Añadir json-server (backend falso)

En la raíz del proyecto `ciudades-app/` crea un fichero `db.json` con contenido inicial:

`db.json`

```
{
  "cities": [
    { "id": 1, "name": "Madrid" },
    { "id": 2, "name": "Barcelona" }
  ]
}
```

Ejecuta (en otra terminal) el servidor json-server:

```
npx json-server --watch db.json --port 3000
```

Eso expondrá endpoints REST tipo:

- `GET http://localhost:3000/cities`
- `POST http://localhost:3000/cities`
- `PUT http://localhost:3000/cities/:id`
- `DELETE http://localhost:3000/cities/:id`
(json-server escribe los cambios en `db.json`, por lo que la persistencia “ligera” está garantizada entre reinicios de json-server).

Paso 2 — Preparar Angular (módulos importantes)

Abrir `src/app/app.module.ts` e importa `HttpClientModule` y `ReactiveFormsModule`:

```
import { BrowserModule } from '@angular/platform-browser';
import { NgModule } from '@angular/core';
import { HttpClientModule } from '@angular/common/http';    // para HttpClient
import { ReactiveFormsModule } from '@angular/forms';        // para formularios reactivos
```

```
import { AppComponent } from './app.component';
import { CityFormComponent } from './city-form/city-form.component';
import { CitiesListComponent } from './cities-list/cities-list.component';
```

```
@NgModule({
  declarations: [
    AppComponent,
    CityFormComponent,
    CitiesListComponent
  ],
  imports: [
    BrowserModule,
    HttpClientModule,
    ReactiveFormsModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Nota: en Angular 18+ aparece la nueva forma `provideHttpClient()`; si trabajas con Angular 18 usa la API moderna (pero el ejemplo con `HttpClientModule` funciona bien en Angular 17 y es didáctico).

Paso 3 — Modelo TypeScript

src/app/models/city.model.ts

```
export interface City {  
  id?: number; // json-server asigna id automáticamente al POST  
  name: string;  
}
```

Paso 4 — Servicio de acceso a datos (CityService)

src/app/services/city.service.ts

```
import { Injectable } from '@angular/core';  
import { HttpClient } from '@angular/common/http';  
import { Observable } from 'rxjs';  
import { map } from 'rxjs/operators';  
import { City } from '../models/city.model';  
  
@Injectable({ providedIn: 'root' })  
export class CityService {  
  private baseUrl = 'http://localhost:3000/cities';  
  
  constructor(private http: HttpClient) {}  
  
  getCities(): Observable<City[]> {  
    return this.http.get<City[]>(this.baseUrl);  
  }  
  
  addCity(city: City): Observable<City> {  
    return this.http.post<City>(this.baseUrl, city);  
  }  
  
  updateCity(city: City): Observable<City> {  
    return this.http.put<City>(`${this.baseUrl}/${city.id}`, city);  
  }  
  
  deleteCity(id: number): Observable<void> {  
    return this.http.delete<void>(`${this.baseUrl}/${id}`);  
  }  
  
  // Comprobación de existencia por nombre (case-insensitive)  
  existsByName(name: string): Observable<boolean> {  
    return this.getCities().pipe(  
      map(list => list.some(c => c.name.trim().toLowerCase() === name.trim().toLowerCase()))  
    );  
  }  
}
```

```
    );  
  }  
}
```

En este ejemplo demostramos cómo encapsular las operaciones CRUD en un servicio — equivalente a un DAO en backend — y además mostramos cómo implementar una comprobación *case-insensitive* consultando y comparando en cliente (json-server no soporta búsquedas case-insensitive out-of-the-box).

Paso 5 — Componente formulario (añadir ciudad, chequeo duplicado)

`src/app/city-form/city-form.component.ts`

```
import { Component } from '@angular/core';  
import { FormBuilder, FormGroup, Validators } from '@angular/forms';  
import { CityService } from '../services/city.service';
```

```
@Component({  
  selector: 'app-city-form',  
  templateUrl: './city-form.component.html'  
})
```

```
export class CityFormComponent {  
  form: FormGroup;  
  loading = false;
```

```
  constructor(private fb: FormBuilder, private cityService: CityService) {  
    this.form = this.fb.group({  
      name: ['', [Validators.required, Validators.minLength(1)]]  
    });  
  }
```

```
  submit(): void {  
    if (this.form.invalid) return;
```

```
    const name = this.form.value.name.trim();  
    this.loading = true;
```

```
    this.cityService.existsByName(name).subscribe(exists => {  
      if (exists) {  
        alert('La ciudad ya existe (no se distingue mayúsc./min.).');  
        this.loading = false;  
        return;  
      }  
    })
```

```

    this.cityService.addCity({ name }).subscribe({
      next: () => { this.form.reset(); this.loading = false; window.location.reload(); },
      error: (err) => { console.error(err); this.loading = false; alert('Error al guardar'); }
    });
  });
}
}

```

`src/app/city-form/city-form.component.html`

```

<form [formGroup]="form" (ngSubmit)="submit()">
  <input formControlName="name" placeholder="Nombre ciudad" />
  <button type="submit" [disabled]="loading || form.invalid">Añadir ciudad</button>
</form>

```

En este caso hacemos uso de **ReactiveFormsModule** para validación y control del formulario.

Paso 6 — Componente lista y export CSV

`src/app/cities-list/cities-list.component.ts`

```

import { Component, OnInit } from '@angular/core';
import { CityService } from '../services/city.service';
import { City } from '../models/city.model';

@Component({
  selector: 'app-cities-list',
  templateUrl: './cities-list.component.html'
})
export class CitiesListComponent implements OnInit {
  cities: City[] = [];

  constructor(private cityService: CityService) {}

  ngOnInit(): void {
    this.load();
  }

  load(): void {
    this.cityService.getCities().subscribe(list => this.cities = list);
  }
}

```

```

deleteCity(c: City): void {
  if (!c.id) return;
  if (!confirm(`Borrar ${c.name}?`)) return;
  this.cityService.deleteCity(c.id).subscribe(() => this.load());
}

exportCsv(): void {
  const csv = this.cities.map(c => `${(c.name || "").replace(/"/g, "\"")}`).join('\n');
  const blob = new Blob([csv], { type: 'text/csv' });
  const url = window.URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = 'ciudades.csv';
  a.click();
  window.URL.revokeObjectURL(url);
}
}

```

<src/app/cities-list/cities-list.component.html>

```

<table *ngIf="cities.length">
  <thead>
    <tr><th>ID</th><th>Ciudad</th><th>Acciones</th></tr>
  </thead>
  <tbody>
    <tr *ngFor="let c of cities">
      <td>{{c.id}}</td>
      <td>{{c.name}}</td>
      <td>
        <button (click)="deleteCity(c)">Borrar</button>
      </td>
    </tr>
  </tbody>
</table>

<button (click)="exportCsv()">Exportar CSV</button>

```

Paso 7 — Ejecutar la app

Terminal 1: arranca json-server

```
# desde la carpeta ciudades-app (donde está db.json)
npx json-server --watch db.json --port 3000
```

1.

Terminal 2: arranca Angular

```
ng serve --open
```

2.

3. Abrir <http://localhost:4200> — debes ver la app (formulario + listado).

- Cuando añades una ciudad, el servicio POST escribe en `db.json`.
- Cuando exportas CSV, se descarga un fichero con los nombres.

Notas a tener en cuenta

- **CORS:** json-server por defecto permite peticiones desde `localhost`, así que no suele haber problemas en dev; en entornos reales hay que habilitar CORS en el backend.
- **Duplicados case-insensitive:** json-server no tiene búsqueda case-insensitive por defecto → lo sencillo para la práctica es comprobar en cliente (como hemos hecho). Explica a los alumnos que en un backend real conviene validar también server-side para evitar condiciones de carrera.
- **Persistencia real:** json-server persiste en `db.json` pero no es producción; para producción usar SGBD (Postgres, MySQL, MongoDB) y un backend adecuado.

Retos para trabajar en clase

1. Implementar edición de ciudad (form con `PUT`).
 2. Mejorar la comprobación de duplicados haciendo una petición al backend que busque por `name` (si el backend lo soporta) o añadiendo una ruta custom en un servidor real.
 3. Añadir paginación y búsqueda remota (query params).
 4. Integrar Angular Material y mejorar la UI (paginador, snackbars).
 5. Implementar guardado local (IndexedDB) cuando json-server no está disponible, y sincronización cuando vuelve (offline-first).
 6. Añadir tests unitarios para `CityService` usando `HttpTestingController`.
-

Referencias (documentación oficial / tutoriales útiles)

- Instalación y CLI Angular (`ng new`, `ng serve`).
- HttpClient (hacer peticiones HTTP, tipado, interceptors).
- Reactive forms (formularios reactivos).
- json-server (tutorial y uso rápido `--watch db.json`).

Ejemplo CRUD Ciudades con Angular 18 + json-server

1. Crear el proyecto con Angular 18

```
npx @angular/cli@18 new ciudades-app --standalone --style=css --routing=false
cd ciudades-app
```

- `--standalone` → crea una aplicación sin `AppModule`, usando `standalone components`, que es el estilo moderno.
-

2. Añadir json-server

En la raíz del proyecto crea `db.json`:

```
{
  "cities": [
    { "id": 1, "name": "Madrid" },
    { "id": 2, "name": "Barcelona" }
  ]
}
```

Lanza el servidor:

```
npx json-server --watch db.json --port 3000
```

3. Configuración de `main.ts`

Abre `src/main.ts` y añade los providers necesarios:

```
import { bootstrapApplication } from '@angular/platform-browser';
import { provideHttpClient } from '@angular/common/http';
import { provideAnimations } from '@angular/platform-browser/animations';
import { AppComponent } from './app/app.component';
```

```
bootstrapApplication(AppComponent, {
```

```
providers: [  
  provideHttpClient(),  
  provideAnimations()  
]  
}).catch(err => console.error(err));
```

4. Modelo City

`src/app/models/city.model.ts`

```
export interface City {  
  id?: number;  
  name: string;  
}
```

5. Servicio CityService

`src/app/services/city.service.ts`

```
import { Injectable } from '@angular/core';  
import { HttpClient } from '@angular/common/http';  
import { Observable, map } from 'rxjs';  
import { City } from '../models/city.model';  
  
@Injectable({ providedIn: 'root' })  
export class CityService {  
  private baseUrl = 'http://localhost:3000/cities';  
  
  constructor(private http: HttpClient) {}  
  
  getCities(): Observable<City[]> {  
    return this.http.get<City[]>(this.baseUrl);  
  }  
  
  addCity(city: City): Observable<City> {  
    return this.http.post<City>(this.baseUrl, city);  
  }  
  
  deleteCity(id: number): Observable<void> {  
    return this.http.delete<void>(`${this.baseUrl}/${id}`);  
  }  
}
```

```

existsByName(name: string): Observable<boolean> {
  return this.getCities().pipe(
    map(list => list.some(c => c.name.trim().toLowerCase() === name.trim().toLowerCase()))
  );
}
}

```

6. Formulario para añadir ciudades

Generamos un componente standalone:

ng generate component city-form --standalone

`src/app/city-form/city-form.component.ts`

```

import { Component } from '@angular/core';
import { FormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';
import { CityService } from '../services/city.service';
import { CommonModule } from '@angular/common';

```

```

@Component({
  selector: 'app-city-form',
  standalone: true,
  imports: [ReactiveFormsModule, CommonModule],
  templateUrl: './city-form.component.html'
})

```

```

export class CityFormComponent {
  form = this.fb.group({
    name: ['', Validators.required]
  });
}

```

```
loading = false;
```

```
constructor(private fb: FormBuilder, private cityService: CityService) {}
```

```

submit(): void {
  if (this.form.invalid) return;
  const name = this.form.value.name!.trim();
  this.loading = true;
}

```

```

this.cityService.existsByName(name).subscribe(exists => {
  if (exists) {
    alert('La ciudad ya existe (ignora mayúsc/minúsc).');
    this.loading = false;
  }
});

```

```

    return;
  }

  this.cityService.addCity({ name }).subscribe({
    next: () => {
      this.form.reset();
      this.loading = false;
      window.location.reload();
    },
    error: () => {
      this.loading = false;
      alert('Error al guardar');
    }
  });
});
}
}

```

`src/app/city-form/city-form.component.html`

```

<form [formGroup]="form" (ngSubmit)="submit()">
  <input type="text" formControlName="name" placeholder="Ciudad" />
  <button type="submit" [disabled]="loading || form.invalid">Añadir</button>
</form>

```

7. Lista de ciudades + export CSV

Generamos otro componente:

`ng generate component cities-list --standalone`

`src/app/cities-list/cities-list.component.ts`

```

import { Component, OnInit } from '@angular/core';
import { CityService } from '../services/city.service';
import { City } from '../models/city.model';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-cities-list',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './cities-list.component.html'
})

```

```

export class CitiesListComponent implements OnInit {
  cities: City[] = [];

  constructor(private cityService: CityService) {}

  ngOnInit(): void {
    this.load();
  }

  load(): void {
    this.cityService.getCities().subscribe(list => this.cities = list);
  }

  deleteCity(c: City): void {
    if (!c.id) return;
    if (confirm(`Borrar ${c.name}?`)) {
      this.cityService.deleteCity(c.id).subscribe(() => this.load());
    }
  }

  exportCsv(): void {
    const csv = this.cities.map(c => `${(c.name || "").replace(/"/g, '"')}`).join("\n");
    const blob = new Blob([csv], { type: 'text/csv' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = 'ciudades.csv';
    a.click();
    URL.revokeObjectURL(url);
  }
}

```

[src/app/cities-list/cities-list.component.html](#)

```

<table *ngIf="cities.length">
  <thead>
    <tr><th>ID</th><th>Ciudad</th><th>Acciones</th></tr>
  </thead>
  <tbody>
    <tr *ngFor="let c of cities">
      <td>{{c.id}}</td>
      <td>{{c.name}}</td>
      <td><button (click)="deleteCity(c)">Borrar</button></td>
    </tr>
  </tbody>
</table>

```

```
<button (click)="exportCsv()">Exportar CSV</button>
```

8. AppComponent

`src/app/app.component.ts`

```
import { Component } from '@angular/core';
import { CityFormComponent } from '../city-form/city-form.component';
import { CitiesListComponent } from '../cities-list/cities-list.component';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CityFormComponent, CitiesListComponent],
  template: `
    <h1>Gestión de Ciudades</h1>
    <app-city-form></app-city-form>
    <app-cities-list></app-cities-list>
  `,
})
export class AppComponent {}
```

9. Ejecutar la aplicación

En dos terminales:

```
# Terminal 1 (json-server)
npx json-server --watch db.json --port 3000
```

```
# Terminal 2
ng serve --open
```

En `http://localhost:4200` podrás:

- Añadir ciudades (sin duplicados case-insensitive).
- Listarlas y borrarlas.
- Exportarlas a CSV.

¿Por qué Angular usa TypeScript y no JavaScript?

1. Tipado estático y fuerte

- En JavaScript todo es dinámico: una variable puede cambiar de tipo en tiempo de ejecución.
- En TypeScript declaras los **tipos de datos** (string, number, boolean, interfaces, clases).
- Ventaja: el compilador detecta errores antes de ejecutar el código (desarrollo más seguro).

Ejemplo:

```
let nombre: string = "Madrid";  
nombre = 123; // ❌ Error en tiempo de compilación
```

2. Mejor experiencia en el IDE (IntelliSense, autocompletado)

- Gracias a los tipos, VSCode y otros editores muestran sugerencias, documentación inline y autocompletado.
 - Los usuarios ven “qué métodos” tienen disponibles sin tener que memorizar todo.
-

3. Orientación a objetos real

- JavaScript tiene clases desde ES6, pero TypeScript amplía con:
 - Interfaces
 - Modificadores de acceso (`public`, `private`, `protected`)
 - Genéricos
 - Decoradores (que en Angular se usan para `@Component`, `@Injectable`, etc.)

Ejemplo con interfaz:

```
interface Ciudad {  
  id: number;  
  nombre: string;  
}
```

```
const ciudad: Ciudad = { id: 1, nombre: "Madrid" };
```

4. Escalabilidad en proyectos grandes

- Angular está pensado para equipos grandes, con cientos de ficheros.
 - TypeScript permite **contratos claros** entre módulos y servicios.
 - Si un alumno se equivoca al pasar un objeto, el compilador lo avisa al instante.
-

5. Compatibilidad con JavaScript

- Todo código JavaScript válido también lo es en TypeScript.
- Angular compila a JavaScript al final, así que se ejecuta en cualquier navegador.
- Es decir: **TypeScript es un superconjunto de JavaScript** → no quita nada, solo añade seguridad y herramientas.

6. Decoradores y metaprogramación

- Angular hace uso intensivo de **decoradores** (`@Component`, `@Injectable`, `@NgModule` o en Angular 18, los `@Component` standalone).
 - Los decoradores son una extensión de TypeScript, no estándar en JavaScript puro (todavía).
 - Esto hace que Angular dependa de TypeScript para ofrecer esa arquitectura basada en anotaciones.
-

Conclusión

- **JavaScript** → lenguaje dinámico, flexible, pero poco seguro en proyectos grandes.
- **TypeScript** → añade **tipado, orientación a objetos, autocompletado, decoradores y detección de errores temprana**.
- Angular lo adopta porque está pensado para **proyectos grandes y equipos**: con TypeScript se reduce el riesgo de bugs y se mejora la productividad.